MONITORING · DOMAIN 5 - MONITORING AND ALERTING

Event logs, alerts and notifications

Section 5. Four objectives, and one question: how do you find out something went wrong?

SECTION 5 of 10 WEIGHT 10% TOPIC Alerting

admin@sthithapragna.com

Monitoring is what you can go and look at.
Alerting is what comes and finds you.
Production needs both.

Everything a pipeline did, in one table

- Contains **audit logs, data quality checks, pipeline progress and data lineage**. All four.
- Readable from the pipelines UI, the pipelines API, or by querying it directly as a table.
- The **details** field holds JSON, parsed with the colon operator.
- In Unity Catalog, views support streaming, so the event log can be **read as a stream**.

> ON THE EXAM
>
> Expectation metrics from module 14 live here. This is where you go to see whether quality is degrading over weeks rather than in one run.

Hidden by default, and owned by one person

- By default it is a **hidden Delta table** in the pipeline's default catalog and schema.
- Named **event log, then the pipeline ID**, with the dashes replaced by underscores.
- By default **only the pipeline owner can query it**.
- You can **publish it** from the pipeline's Advanced settings, which makes it an ordinary named table.

> ON THE EXAM
>
> Hidden is not the same as inaccessible. Sufficiently privileged users can query it; the default is simply that only the owner is.

Things that will break a pipeline

- **Do not delete the event log**, or the catalog or schema it was published to.
- Doing so **may cause the pipeline to fail on future updates**. It is not merely a loss of history.
- The event log location also serves as the **schema location for any Auto Loader queries** in that pipeline.
- So a tidy-up that removes it takes the Auto Loader schema tracking with it.

> ON THE EXAM
>
> That second point explains the first. The event log location is doing more work than its name suggests.

Create a view before granting access

- Databricks **recommends creating a view** over the event log table before modifying privileges.
- Some compute settings may let users reach **schema metadata** if the table itself is shared directly.
- The view is also what the documented example queries are written against.
- That is the same instinct as a masking view in module 15: expose the result, not the object.

> ON THE EXAM
>
> A recommendation with a security reason behind it, which makes it more likely to appear as an answer than a style preference would.

The event log table-valued function

- There is an **event log function** that returns the log for a pipeline or a streaming table.
- It **only applies to Unity Catalog pipelines using legacy publishing mode**. Otherwise, query the table.
- It can be called **only by the owner**, and a view built on it **cannot be shared**.
- So for a modern pipeline, the answer is the published table rather than the function.

> ON THE EXAM
>
> This is a function you will meet in older material and mostly should not use. Recognise it, and know it does not apply to current pipelines.

Event hooks and the account-wide view

- **Event hooks** define custom actions to run when events are logged, such as sending an alert.
- There is also a **system table** holding pipeline event log entries for the whole region.
- That is the route when you want one query across every pipeline rather than one at a time.
- Remember its retention and delivery caveats from module 17. It is not real-time.

> ON THE EXAM
>
> Per-pipeline event log for depth on one pipeline. System table for breadth across all of them.

A query, a condition, and a notification

- Now **generally available**. Authored in the SQL editor, evaluated on a schedule or inside a job.
- Databricks names the uses directly: a table **not refreshed within an hour**, row counts below baseline, null rates above a threshold.
- Also duplicate keys, distribution shift, and business-metric drift.
- So this is the mechanism for **data** quality, as opposed to **job** outcomes.

> ON THE EXAM
>
> Expectations catch bad records as they pass. Alerts catch the absence of records, and conditions that only appear in aggregate.

The SQL alert task, and its status

- A **SQL alert task** evaluates an alert as a step in a job, on a warehouse you specify.
- The task reports **Succeeded whenever the alert evaluated successfully**, whether or not the condition triggered.
- **Failed** means an evaluation error, such as a warehouse connectivity problem or a bad query.
- It evaluates the alert **independently of any schedule** set on the alert itself.

> ON THE EXAM
>
> Task status tells you whether the check ran, not what it found. That distinction is exactly the kind of thing a professional paper tests.

How the alert outcome reaches the rest of the job

- The task exposes an **alert state** value, compared against **TRIGGERED**.
- An **If/else task** reads that and branches, which is module 9's control flow applied to data quality.
- So you can halt publication, or run a remediation path, when a check fails.
- Relying on task status instead would branch on whether the check ran, not on its verdict.

> ON THE EXAM
>
> This is the pattern the objective is really pointing at: a quality gate that stops bad data reaching consumers.

Five triggers

TRIGGER	FIRES WHEN
On start	The run begins.
On success	The run completes successfully.
On failure	Internal error, or a failed or timed-out result.
Duration warning	The run exceeds the configured run duration threshold.
Streaming backlog	A streaming backlog threshold is exceeded.

ON THE EXAM

A maximum of three destinations per trigger. Both email and webhook destinations are available for each.

The health metrics behind the triggers

- Duration warnings use a **run duration in seconds** metric set in the job's health rules.
- Streaming backlog thresholds come in four flavours: **bytes, records, seconds and files**.
- Seconds is usually the one that matches a business requirement, since it expresses lateness.
- A notification without a configured threshold has nothing to fire on.

> ON THE EXAM
>
> Choosing the metric is a modelling decision. Backlog in records tells you volume; backlog in seconds tells you how stale your data is.

How backlog alerting actually behaves

- Alerting is based on the **10-minute average** of the metric, not on an instantaneous spike.
- If the issue persists, notifications are **resent every 30 minutes**.
- That averaging is deliberate. A brief burst will not wake anybody up.
- **Streaming backlog notifications are not supported for pipeline tasks.**

> ON THE EXAM
>
> So a declarative pipeline needs a different route for backlog alerting. The event log carries backlog metrics in its flow progress events.

The API and command line

- The **Jobs API** lists runs and returns run detail, which is how an external system learns a job's outcome.
- The **pipelines API** exposes updates and event log entries in the same way.
- The **command line** wraps both, which is what makes monitoring scriptable from a delivery pipeline.
- Use these when something **outside** Databricks needs to know. Use notifications when a person does.

> ON THE EXAM
>
> The distinction is who is asking. A human wants a notification; an orchestrator wants an API it can poll or a webhook it can receive.

What to carry into the exam

- The event log holds **audit, quality, progress and lineage**. Hidden and owner-only by default.
- **Do not delete it.** It is also the Auto Loader schema location for that pipeline.
- **SQL alerts** monitor the data. **Job notifications** monitor the job. Different objects, different questions.
- A SQL alert task reports **Succeeded when it evaluated**, not when it triggered. Branch on alert state.
- Backlog alerting is a **10-minute average**, resent every 30, and **not supported for pipeline tasks**.

> ON THE EXAM
>
> That completes Section 5 at 10%. Section 6, cost and performance, is next and is the second-largest section at 13%.

> A platform team wants three engineers to monitor data quality trends across a pipeline by querying its event log. The pipeline was created with default settings and none of the three is the owner.
>
> An engineer suggests they each call the event log table-valued function with the pipeline ID.

Why will that not work, and what should they do?

A The function can only be called by the owner. Publish the event log, then create a view and grant on it.

B The function requires DBR 13.3 LTS, so they need to upgrade their compute.

C The event log must be enabled in Advanced settings before any query works.

D They should each be made owner of the pipeline in turn.

◆ PAUSE AND DECIDE

Why will that not work, and what should they do?

A The function can only be called by the owner. Publish the event log, then create a view and grant on it.

Two documented facts: the function is owner-only and its views cannot be shared, and publishing plus a view is the recommended route.

B The function requires DBR 13.3 LTS, so they need to upgrade their compute.

A real requirement, and not the blocker here.

C The event log must be enabled in Advanced settings before any query works.

It exists by default as a hidden table. Publishing changes where it lives, not whether it exists.

D They should each be made owner of the pipeline in turn.

Changes who the pipeline runs as, to solve a read-access problem.

ON THE EXAM

Also note the function applies only to legacy publishing mode. For a current pipeline it is the wrong tool twice over.

> During a workspace clean-up, an old catalog is deleted. It contained the published event log for a production pipeline. Nobody was reading that log and the team considered it disposable.
>
> The next scheduled pipeline update fails.

What is the most likely explanation?

A Deleting the event log breaks the pipeline, and the location also served as the Auto Loader schema location.

B The pipeline cannot start without a place to write its first event, but it will recover on the next run.

C The catalog deletion revoked the pipeline owner's permissions.

D The pipeline is fine; the failure must be unrelated.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Deleting the event log breaks the pipeline, and the location also served as the Auto Loader schema location.

The docs warn against deleting it for exactly this reason, and the location does double duty as the schema location.

B The pipeline cannot start without a place to write its first event, but it will recover on the next run.

Half right on the cause, and wrong that it self-heals.

C The catalog deletion revoked the pipeline owner's permissions.

Permissions are not the mechanism described in the warning.

D The pipeline is fine; the failure must be unrelated.

The documentation explicitly warns that deletion may cause future updates to fail.

ON THE EXAM

An event log looks like history and is partly infrastructure. That is why the warning exists.

A team adds a SQL alert task to a job, checking that a loaded table's row count exceeds a baseline. Downstream tasks depend on the alert task with the default All succeeded condition.

A load arrives with almost no rows. The alert triggers and notifies the team, and the downstream publication tasks run anyway.

Why did publication not stop?

A The alert task reports Succeeded whenever it evaluates, whether or not the condition triggered.

B Run-if conditions do not apply to SQL alert tasks.

C The alert's own schedule overrode the task evaluation.

D The alert condition was evaluated after the downstream tasks began.

◆ PAUSE AND DECIDE

Why did publication not stop?

A The alert task reports Succeeded whenever it evaluates, whether or not the condition triggered.

Task status reflects whether the check ran, not its verdict. Branch on the alert state with an If/else task instead.

B Run-if conditions do not apply to SQL alert tasks.

They apply normally. The problem is what the status means.

C The alert's own schedule overrode the task evaluation.

The task evaluates independently of any schedule on the alert.

D The alert condition was evaluated after the downstream tasks began.

Dependencies are respected; the alert task completed first.

ON THE EXAM

This is module 9's distinction again: run-if branches on outcome, If/else branches on a value. The alert's verdict is a value.

> A nightly job normally completes in 20 minutes. Occasionally it takes four hours but still succeeds, and nobody notices until a morning dashboard is stale. The team has on-failure notifications configured.
>
> They want to be told when the job runs long, without failing it.

What should they configure?

A. A timeout, so the job fails after a set duration and the on-failure notification fires.

B. A run duration health threshold with a duration warning notification.

C. A SQL alert on the job's runtime in the system tables.

D. On-start notifications, so the team can time it themselves.

◆ PAUSE AND DECIDE

What should they configure?

A. A timeout, so the job fails after a set duration and the on-failure notification fires.

Meets the notification need by killing a job that would have finished. The question said without failing it.

B. A run duration health threshold with a duration warning notification.

Exactly what the trigger exists for: notify on a long run while letting it complete.

C. A SQL alert on the job's runtime in the system tables.

Would work, with system table delivery latency on top and a mechanism built by hand.

D. On-start notifications, so the team can time it themselves.

Notifies at the wrong moment and asks a human to be the monitor.

ON THE EXAM

Read the constraint: without failing it. That single phrase rules out the timeout answer that otherwise looks reasonable.

A declarative pipeline ingests from Kafka. The team wants to be notified when the pipeline falls more than ten minutes behind. They add a streaming backlog notification with a backlog seconds threshold to the job that runs the pipeline task.

The notification never fires, even during a confirmed two-hour backlog.

What is the explanation?

A Backlog alerting uses a 10-minute average, so a two-hour backlog is averaged away.

B Streaming backlog notifications are not supported for pipeline tasks. Use the event log's flow progress backlog metrics instead.

C Backlog seconds is not a supported metric; only bytes and records are.

D The threshold must be set on the pipeline rather than in the job's health rules.

◆ PAUSE AND DECIDE

What is the explanation?

A Backlog alerting uses a 10-minute average, so a two-hour backlog is averaged away.

The averaging smooths brief spikes. A sustained two-hour backlog would clear any average.

B Streaming backlog notifications are not supported for pipeline tasks. Use the event log's flow progress backlog metrics instead.

A documented limitation, and the event log carries backlog metrics in flow progress events.

C Backlog seconds is not a supported metric; only bytes and records are.

All four exist: bytes, records, seconds and files.

D The threshold must be set on the pipeline rather than in the job's health rules.

Health rules are the right place. The task type is the problem.

ON THE EXAM

The configuration is correct and the task type is unsupported. Nothing errors, which is why this is hard to spot.

> An external orchestration system outside Databricks must know whether a nightly job succeeded before it starts downstream work in another platform. Email notifications already go to the data team.
>
> An engineer proposes parsing the notification emails.

What is the better approach?

A Parse the emails, since they already contain the outcome.

B Use the Jobs API or a webhook notification, so the external system receives or retrieves the outcome directly.

C Grant the external system access to the workspace UI.

D Have the job write a status file to cloud storage that the other system polls.

◆ PAUSE AND DECIDE

What is the better approach?

A. Parse the emails, since they already contain the outcome.

Builds an integration on a human-readable format that can change without notice.

B. Use the Jobs API or a webhook notification, so the external system receives or retrieves the outcome directly.

The documented route for programmatic monitoring. A webhook pushes; the API is polled.

C. Grant the external system access to the workspace UI.

A UI is not an integration point.

D. Have the job write a status file to cloud storage that the other system polls.

Workable, and reinvents an interface that already exists, with a new failure mode if the write is missed.

ON THE EXAM

Ask who needs to know. A person gets a notification; a system gets an API or a webhook.